

Journal de Bord – Semaine 3

Nom : PYRCZAK Noah

Entreprise : CIMPA – Sopra Steria

Période : Semaine 3 du stage

Description des tâches effectuées

Cette semaine, j'ai développé un script PowerShell permettant de surveiller automatiquement l'état de certains services critiques sur une machine Windows. Le script vérifie si les services (comme IIS, SQL Server, Spooler) sont actifs. S'ils sont arrêtés, il les relance et génère un rapport avec les statuts. L'objectif était d'améliorer la disponibilité automatique des services locaux.

Compétences mobilisées : C1.1.1, C1.1.2, C1.2.1, C1.3.1, C1.3.3, C1.4.1, C1.4.2

Problèmes rencontrés / Solutions apportées

- Difficulté à identifier les noms exacts des services système (solution : utilisation de Get-Service)
- Droits requis pour redémarrer un service : exécution du script en tant qu'administrateur
- Vérification de la fiabilité des relances : mise en place d'une boucle de log

Fiche de Réalisation Professionnelle n°3

– Semaine 3

Nom : PYRCZAK Noah

Entreprise : CIMPA – Sopra Steria

Sujet : Script PowerShell de supervision de services Windows

Contexte : Projet en autonomie visant à détecter automatiquement les services critiques arrêtés et les relancer.

Objectif

Assurer la disponibilité des services en production via un contrôle et une relance automatique par script.

Activités réalisées

- Liste des services critiques à surveiller
- Développement du script de test et relance automatique
- Génération d'un fichier de rapport horodaté
- Tests sur machine virtuelle

Compétences mobilisées (Bloc 1)

C1.1.1 / C1.1.2 – Identifier les ressources et normes

C1.2.1 – Adapter un composant logiciel

C1.3.1 / C1.3.3 – Tester un service et résoudre les incidents

C1.4.1 / C1.4.2 – Automatiser le déploiement

Script PowerShell utilisé

```
$servicesAControle = @(
```

```
"W3SVC",
```

```

"MSSQLSERVER",

"Spooler",

"BITS",

"Dhcp",

"TermService",

"LanmanWorkstation", # Service Workstation

"Netlogon",    # Service d'ouverture de session réseau

"Dnscache",    # Client DNS

"RpcSs",       # Appelle de procédure distante (RPC)

"SysMain",     # Superfetch / SysMain

"Themes",      # Services de Thèmes

"DoSvc",       # Service d'optimisation de livraison

"CertSvc"      # Services de certificat Active Directory (si applicable)

)

$rapport = @()$dateExecutionScript = Get-Date

Write-Host "Initialisation de la surveillance des services Windows..." -ForegroundColor
DarkYellow

Write-Host "Date et heure de début : $($dateExecutionScript.ToString('F'))" -
ForegroundColor DarkGray

# Définition d'une variable pour l'état "Running" afin d'éviter les chaînes magiques

$statutEnCours = 'Running'

foreach ($svc in $servicesAControle) {

    Write-Host "Traitement du service : $($svc.ToUpper())" -ForegroundColor Cyan

    try {

        $etat = Get-Service -Name $svc -ErrorAction Stop
    }

```

```

$tempStatusCheck = $etat.Status

Write-Host "Statut actuel de '$svc' : $($tempStatusCheck)" -ForegroundColor White
}

catch {

    Write-Warning "Le service '$svc' n'a pas été trouvé ou est inaccessible. Ignoré."

    # Création d'une entrée pour les services introuvables

    $ligneErreur = [PSCustomObject]@{

        Service = $svc

        Etat  = "Introuvable/Inaccessible"

        Date  = Get-Date -Format "yyyy-MM-dd HH:mm:ss"

        Relance = "N/A"

    }

    $rapport += $ligneErreur

    continue

}

$ligne = [PSCustomObject]@{

    Service = $svc

    Etat  = $etat.Status

    Date  = Get-Date -Format "yyyy-MM-dd HH:mm:ss"

}

$rapport += $ligne

if ($etat.Status -ne $statutEnCours) {

    Write-Host "Le service '$svc' n'est PAS en cours d'exécution. Tentative de démarrage..."
-ForegroundColor Yellow

    try {

```

```

Start-Service -Name $svc -ErrorAction Stop

$ligne.Relance = "Tentative de relance effectuée"

Write-Host "Le service '$svc' a été démarré avec succès." -ForegroundColor Green
}

catch {

    $ligne.Relance = "Échec de la relance"

    Write-Error "Échec du démarrage du service '$svc'. Erreur: $($_.Exception.Message)"

}

} else {

    Write-Host "Le service '$svc' est déjà en cours d'exécution. OK." -ForegroundColor
Green

    $ligne.Relance = "OK"

}

Write-Host ""

}

Write-Host "--- Rapport de Surveillance des Services ---" -ForegroundColor White -
BackgroundColor DarkBlue

Write-Host ""

$rapport | Format-Table -AutoSize

Write-Host ""

$cheminBureau = [Environment]::GetFolderPath("Desktop")

$nomFichier = "RapportServices_$(Get-Date -Format 'yyyyMMdd_HH:mm:ss').txt"

$cheminLog = Join-Path $cheminBureau $nomFichier

# Une variable pour confirmer le succès de l'écriture

$fichierEcritAvecSucces = $false

```

```
try {  
    $rapport | Out-File -FilePath $cheminLog -Encoding UTF8 -Force  
    $fichierEcritAvecSucces = $true  
}  
catch {  
    Write-Error "Échec de l'enregistrement du rapport dans le fichier '$cheminLog'. Erreur:  
    $($_.Exception.Message)"  
}  
if ($fichierEcritAvecSucces) {  
    Write-Host "Rapport généré et sauvegardé avec succès :" -ForegroundColor Green  
    Write-Host $cheminLog -ForegroundColor Green  
} else {  
    Write-Host "Le rapport n'a pas pu être sauvegardé. Vérifiez les erreurs ci-dessus." -  
    ForegroundColor Red  
}  
$dateFinExecution = Get-Date  
$dureeExecution = $dateFinExecution - $dateExecutionScript  
Write-Host ""  
Write-Host "--- Surveillance des services terminée ---" -ForegroundColor White -  
    BackgroundColor DarkBlue  
Write-Host "Temps d'exécution total : $($dureeExecution.TotalSeconds) secondes." -  
    ForegroundColor DarkGray
```